

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami :

1. Memahami konsep dasar *Transaction Control Language* (TCL) dalam basis data relasional.
2. Memahami penggunaan COMMIT, ROLLBACK, dan SAVEPOINT dalam mengelola perubahan data.
3. Memahami penerapan TCL pada proses INSERT, UPDATE, dan DELETE data.
4. Memahami konsep *read consistency* dalam menjaga konsistensi data saat transaksi berlangsung.
5. Memahami fungsi *locks* dalam mengatur akses data dan mencegah konflik antar transaksi.

12.2 Dasar Teori

Transaction Control Language (TCL) adalah bagian dari SQL yang digunakan untuk mengatur transaksi dalam basis data relasional. TCL mencakup perintah COMMIT, ROLLBACK, dan SAVEPOINT yang berfungsi mengontrol perubahan data sebelum disimpan secara permanen. Perintah ini penting agar proses perubahan data dapat dikendalikan dengan aman dan konsisten. TCL juga membantu menjaga integritas data ketika terjadi kesalahan dalam proses perubahan data. Dengan TCL, pengguna dapat menentukan apakah perubahan data perlu disimpan, dibatalkan, atau dikembalikan ke titik tertentu dalam transaksi (Oracle, 2020).

COMMIT digunakan untuk menyimpan seluruh perubahan data dalam transaksi secara permanen ke *database*. Perintah ini biasanya digunakan setelah proses INSERT, UPDATE, atau DELETE selesai dilakukan dengan benar. Dengan COMMIT, perubahan data tidak dapat dibatalkan lagi menggunakan ROLLBACK biasa. Penggunaan COMMIT membantu memastikan bahwa data yang sudah valid benar-benar tersimpan dalam sistem basis data (Beaulieu, 2020).

ROLLBACK digunakan untuk membatalkan perubahan data yang belum disimpan secara permanen. Perintah ini berguna ketika terjadi kesalahan dalam proses INSERT, UPDATE, atau DELETE, sehingga *database* dapat kembali ke kondisi sebelum transaksi dilakukan. ROLLBACK membantu menjaga konsistensi dan integritas data dalam sistem basis data (Oracle, 2020).

SAVEPOINT adalah titik pemulihan yang dibuat di dalam *transaction*. Dengan SAVEPOINT, pengguna dapat membatalkan sebagian perubahan tanpa harus membatalkan seluruh *transaction*. Perintah ini berguna ketika beberapa proses DML dilakukan bertahap dan hanya bagian tertentu yang perlu dikembalikan (Beaulieu, 2020).

Locks adalah mekanisme penguncian data yang digunakan *database* untuk mengatur akses bersamaan antar *transaction*. *Locks* mencegah dua transaksi mengubah data yang sama secara bersamaan sehingga risiko konflik dan inkonsistensi dapat dikurangi. Mekanisme ini mendukung keamanan proses *transaction*, terutama saat menjalankan perintah INSERT, UPDATE, dan DELETE (Beaulieu, 2020).

12.3 Hasil dan Pembahasan

Adapun beberapa percobaan *query* dari modul *section* 18 yang sudah dipelajari yaitu sebagai berikut:

1. **Query:**

```
UPDATE copy_departments SET manager_id= 101  
WHERE department_id = 60;
```

Hasil:

```
SQL> UPDATE copy_departments SET manager_id = 101 WHERE department_id = 60;  
1 row updated.
```

Gambar 12.1 Tampilan hasil pembaruan data `manager_id` pada tabel `copy_departments`

Penjelasan:

Query tersebut digunakan untuk memperbarui data pada tabel `copy_departments`. Perintah `UPDATE` ini mengubah nilai kolom `manager_id` menjadi 101 khusus untuk departemen dengan ID 60. Hasil eksekusi menampilkan pesan konfirmasi bahwa satu baris data telah berhasil diperbarui.

2. **Query:**

```
SAVEPOINT one;
```

Hasil:

```
SQL> SAVEPOINT one;  
  
Savepoint created.
```

Gambar 12.2 Tampilan hasil pembuatan `SAVEPOINT` di dalam transaksi

Penjelasan:

Query tersebut digunakan untuk menandai titik tertentu dalam transaksi yang sedang berjalan. Perintah `SAVEPOINT` ini membuat penanda bernama “one” untuk membatalkan sebagian perubahan jika terjadi kesalahan. Hasil eksekusi menampilkan pesan konfirmasi bahwa titik pemulihan tersebut telah berhasil dibuat.

3. **Query:**

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:

```
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id)  
2 VALUES(130, 'Estate Management', 102, 1500);  
1 row created.
```

Gambar 12.3 Tampilan hasil penambahan data baru pada tabel
copy_departments

Penjelasan:

Query tersebut digunakan untuk menambahkan baris data baru ke dalam tabel copy_departments. Perintah INSERT INTO ini memasukkan data departemen “Estate Management” lengkap dengan ID, manajer, dan lokasinya. Hasil eksekusi menampilkan pesan konfirmasi bahwa satu baris data baru telah berhasil ditambahkan.

4. **Query:**

```
UPDATE copy_departments SET department_id = 140;
```

Hasil:

```
SQL> UPDATE copy_departments SET department_id = 140;  
28 rows updated.
```

Gambar 12.4 Tampilan hasil pembaruan pada tabel
copy_departments

Penjelasan:

Query tersebut digunakan untuk memperbarui data pada tabel copy_departments secara menyeluruh. Perintah UPDATE ini mengubah nilai kolom department_id menjadi 140 untuk semua baris data karena tidak menggunakan klausa WHERE. Hasil eksekusi menampilkan pesan konfirmasi bahwa sebanyak 28 baris data telah berhasil diperbarui.

5. **Query:**

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:

```
SQL> ROLLBACK TO SAVEPOINT one;  
  
Rollback complete.
```

Gambar 12.5 Tampilan hasil eksekusi `ROLLBACK TO SAVEPOINT one`

Penjelasan:

Query tersebut digunakan untuk membatalkan perubahan data kembali ke titik penanda yang telah ditentukan sebelumnya. Perintah `ROLLBACK TO SAVEPOINT` ini mengembalikan kondisi *database* ke posisi “one” untuk memulihkan transaksi sebelum terjadinya kesalahan massal. Hasil eksekusi menampilkan pesan konfirmasi bahwa proses pengembalian data telah berhasil diselesaikan.

6. **Query:**

```
COMMIT;
```

Hasil:

```
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.6 Tampilan hasil eksekusi perintah `COMMIT`

Penjelasan:

Query tersebut digunakan untuk menyimpan semua perubahan data secara permanen ke dalam *database*. Perintah `COMMIT` ini memastikan bahwa seluruh rangkaian transaksi yang telah dilakukan berhasil disimpan dan tidak dapat dibatalkan lagi. Hasil eksekusi menampilkan pesan konfirmasi bahwa proses penyimpanan permanen telah selesai dilakukan.

12.4 Latihan

Adapun latihan pada bab ini adalah sebagai berikut:

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO mahasiswa VALUES (24071111, 'Luffy',  
'Rekayasa Mesin');  
  
INSERT INTO mahasiswa VALUES (24072222, 'Nami',  
'Rekayasa Elektro');  
  
SAVEPOINT nami;  
  
UPDATE mahasiswa SET prodi = 'Rekayasa Kimia' WHERE  
nim = 24072222;  
  
ROLLBACK TO nami;  
  
COMMIT;
```

Hasil:

```
SQL> INSERT INTO mahasiswa VALUES (24071111, 'Luffy', 'Rekayasa Mesin');  
1 row created.  
SQL> INSERT INTO mahasiswa VALUES (24072222, 'Nami', 'Rekayasa Elektro');  
1 row created.  
SQL> SAVEPOINT nami;  
Savepoint created.  
SQL> UPDATE mahasiswa SET prodi = 'Rekayasa Kimia' WHERE nim = 24072222;  
1 row updated.  
SQL> ROLLBACK TO nami;  
Rollback complete.  
SQL> COMMIT;  
Commit complete.
```

Gambar 12.7 Tampilan hasil pengelolaan transaksi data mahasiswa menggunakan perintah TCL

Penjelasan:

Query tersebut digunakan untuk mengelola rangkaian operasi manipulasi data pada tabel `mahasiswa` secara terstruktur, aman, dan konsisten menggunakan perintah-perintah TCL (Transaction Control Language). Proses eksekusi ini diawali dengan memasukkan dua baris data mahasiswa baru ke dalam sistem, dilanjutkan dengan pembuatan `SAVEPOINT` bernama “nami” sebagai penanda posisi transaksi. Setelah itu, sistem melakukan pembaruan pada kolom `prodi` tertentu, namun perubahan tersebut akhirnya dibatalkan kembali ke posisi `SAVEPOINT` “nami” melalui perintah `ROLLBACK TO`. Hasil eksekusi membuktikan bahwa hanya data yang valid sebelum `SAVEPOINT` yang dipertahankan, lalu seluruh rangkaian operasi tersebut sukses disimpan secara permanen ke dalam *database* melalui perintah `COMMIT`.

12.5 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa TCL (Transaction Control Language) merupakan bagian krusial dari SQL yang berfungsi untuk mengelola dan mengontrol rangkaian transaksi data di dalam basis data relasional. Sistem basis data memanfaatkan kombinasi perintah `COMMIT`, `ROLLBACK`, dan `SAVEPOINT` untuk memastikan setiap perubahan data akibat operasi DML dapat dikendalikan dengan aman serta konsisten. Implementasi seluruh perintah tersebut sangat penting dalam menjaga integritas dan validitas data, terutama ketika terjadi kesalahan teknis di tengah proses manipulasi data.

Mekanisme pengendalian transaksi ini bekerja secara fleksibel dengan memberikan opsi penyimpanan permanen, pembatalan total, maupun pengembalian data ke titik tertentu yang telah ditandai. Kehadiran fungsi `SAVEPOINT` mempermudah proses pemulihan sebagian data tanpa perlu membatalkan seluruh rangkaian transaksi yang sudah berjalan sukses sejak awal. Selain itu, penerapan sistem *locks* bertindak sebagai pelindung tambahan yang mengatur akses bersamaan antar-transaksi guna mencegah terjadinya konflik dan inkonsistensi data.